Shyamoli Textile Engineering College
Department of Computer Science and Engineering
CSE-4237: Digital Image Processing

Lec-17: DCT Transform

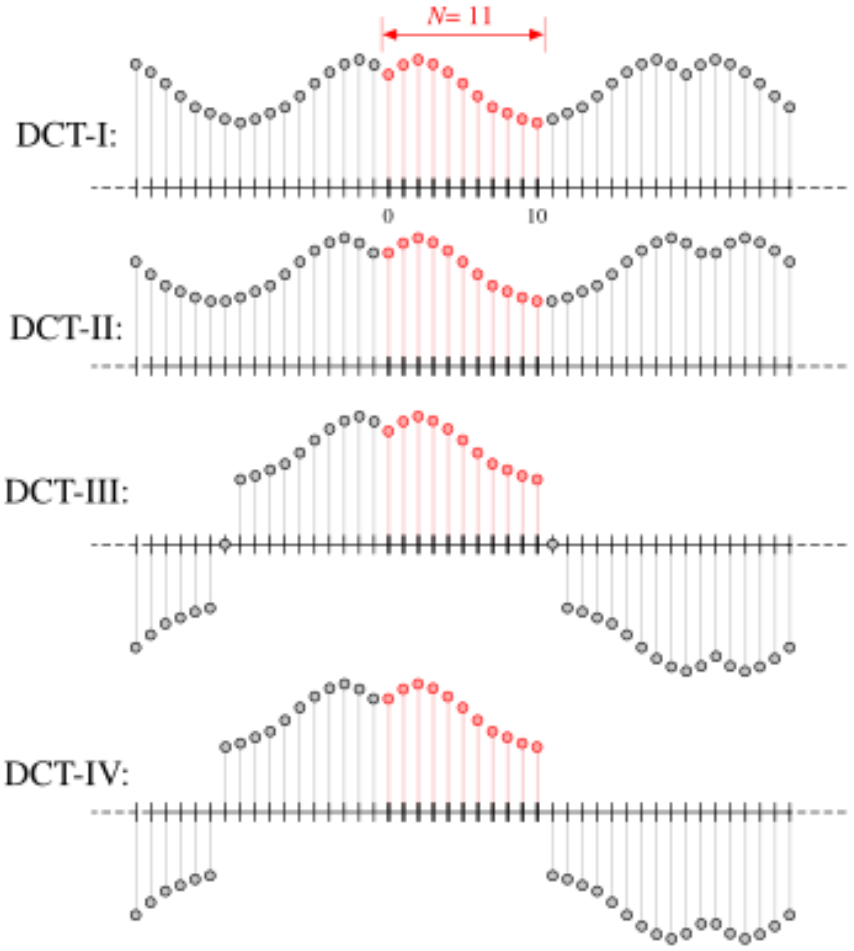
Digital Image Processing by Rafael C. Gonzalez and Richard Eugene Woods (Chapter-7)

Md. Ariful Islam
Assistant Professor
Dept. of Robotics and Mechatronics Engineering
University of Dhaka

1. Discrete Cosine Transform (DCT)

The DCT is a mathematical transformation that *converts a signal (e.g., an image) from the spatial domain into the frequency domain*.

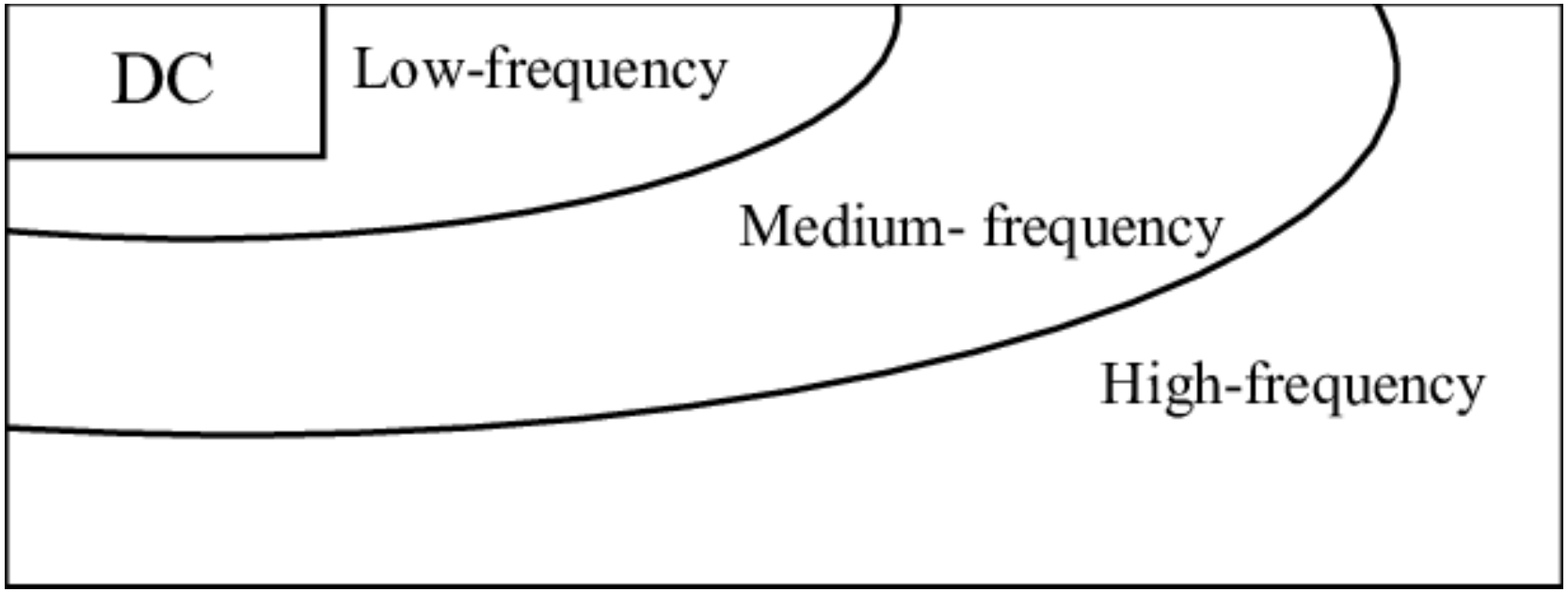
It expresses the image as *a sum of cosine functions oscillating at different frequencies*.



This is useful because it helps *separate the image into parts* of varying importance (low-frequency and high-frequency components).

- **Low-frequency components:** Represent the *overall structure and smooth variations* in the image (*most important for visual perception*).

- **High-frequency components:** Represent *fine details, edges, and noise* (less important for visual perception).

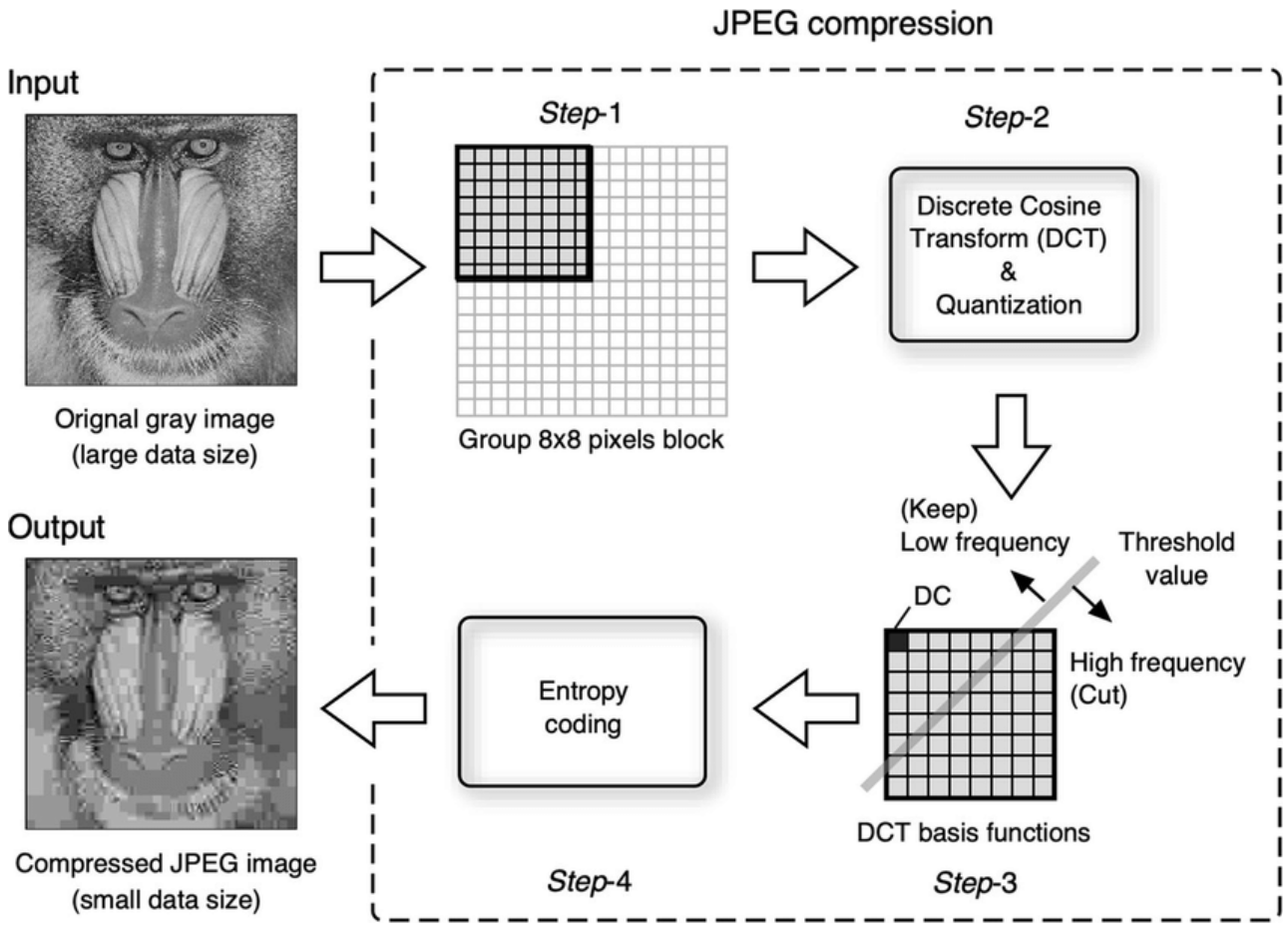


1.1 Why is DCT used in image processing?

1.1.1 Compression

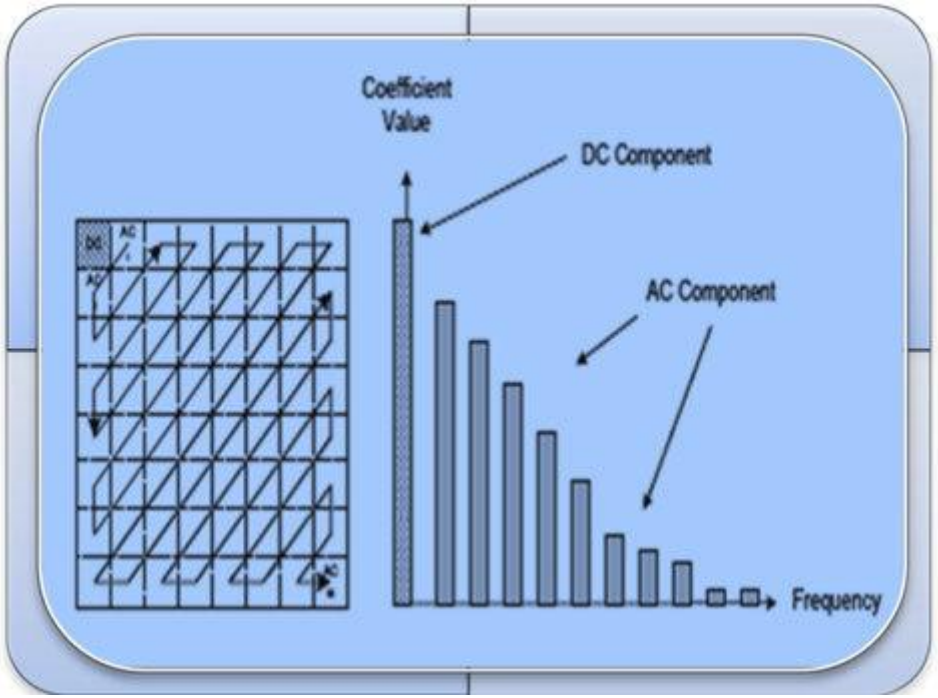
DCT helps in compressing images by *concentrating most of the signal energy into a few low-frequency components*.

- These components can be *stored or transmitted more efficiently.*



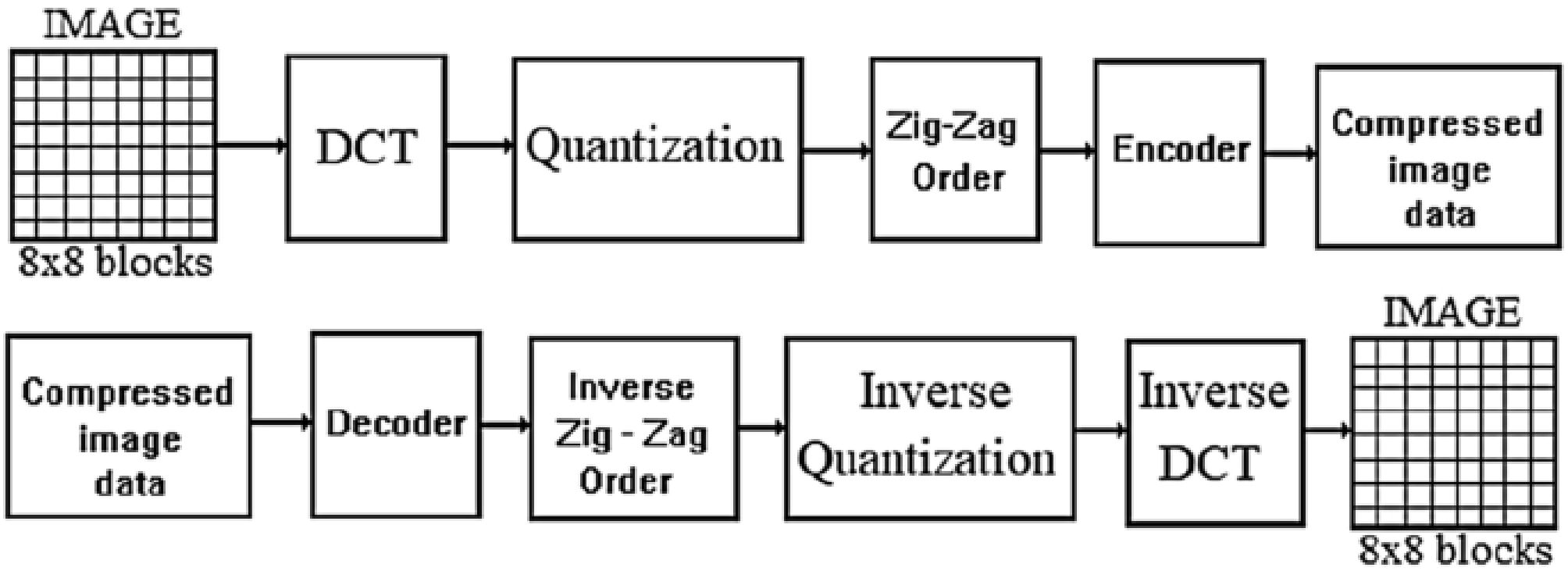
1.1.2 Energy compaction

DCT packs the *most important visual information into a small number of coefficients*, making it *ideal for lossy compression*.



1.2 How does DCT work?

For an image, the *DCT is applied to small blocks* (typically 8x8 pixels).



Step-1: Divide the image into 8x8 blocks:

An image is essentially a *2D array of pixel values*.

- For grayscale images, *each pixel has a single intensity value* (e.g., 0 to 255).
- For color images, *each pixel has three values (e.g., Red, Green, Blue).*

Apply the DCT

To apply the DCT, we *divide the image into smaller, non-overlapping blocks* of 8x8 pixels.

- *Each block is processed independently*.

Example: Grayscale Image

Assume we have a *grayscale image of size 16x16 pixels*. The pixel values are represented as a 2D array.

Original Image (16x16 pixels)

```
Image = [
  [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160],
  [15, 25, 35, 45, 55, 65, 75, 85, 95, 105, 115, 125, 135, 145, 155, 165],
  [20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170],
  [25, 35, 45, 55, 65, 75, 85, 95, 105, 115, 125, 135, 145, 155, 165, 175],
  [30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180],
  [35, 45, 55, 65, 75, 85, 95, 105, 115, 125, 135, 145, 155, 165, 175, 185],
  [40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190],
  [45, 55, 65, 75, 85, 95, 105, 115, 125, 135, 145, 155, 165, 175, 185, 195],
  [50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200],
  [55, 65, 75, 85, 95, 105, 115, 125, 135, 145, 155, 165, 175, 185, 195, 205],
  [60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200, 210],
  [65, 75, 85, 95, 105, 115, 125, 135, 145, 155, 165, 175, 185, 195, 205, 215],
  [70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200, 210, 220],
  [75, 85, 95, 105, 115, 125, 135, 145, 155, 165, 175, 185, 195, 205, 215, 225],
  [80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200, 210, 220, 230],
  [85, 95, 105, 115, 125, 135, 145, 155, 165, 175, 185, 195, 205, 215, 225, 235]
]
```

Since the image is 16x16, it can be *divided into 4 non-overlapping 8x8 blocks*.

Block 1 (Top-left 8x8 block):

```
Block1 = [
  [10, 20, 30, 40, 50, 60, 70, 80],
  [15, 25, 35, 45, 55, 65, 75, 85],
  [20, 30, 40, 50, 60, 70, 80, 90],
  [25, 35, 45, 55, 65, 75, 85, 95],
  [30, 40, 50, 60, 70, 80, 90, 100],
  [35, 45, 55, 65, 75, 85, 95, 105],
  [40, 50, 60, 70, 80, 90, 100, 110],
  [45, 55, 65, 75, 85, 95, 105, 115]
]
```

Block 2 (Top-right 8x8 block):

```
Block2 = [
  [90, 100, 110, 120, 130, 140, 150, 160],
  [95, 105, 115, 125, 135, 145, 155, 165],
  [100, 110, 120, 130, 140, 150, 160, 170],
  [105, 115, 125, 135, 145, 155, 165, 175],
  [110, 120, 130, 140, 150, 160, 170, 180],
  [115, 125, 135, 145, 155, 165, 175, 185],
  [120, 130, 140, 150, 160, 170, 180, 190],
  [125, 135, 145, 155, 165, 175, 185, 195]
]
```

Block 3 (Bottom-left 8x8 block):

```
Block3 = [
  [50, 60, 70, 80, 90, 100, 110, 120],
  [55, 65, 75, 85, 95, 105, 115, 125],
  [60, 70, 80, 90, 100, 110, 120, 130],
  [65, 75, 85, 95, 105, 115, 125, 135],
  [70, 80, 90, 100, 110, 120, 130, 140],
  [75, 85, 95, 105, 115, 125, 135, 145],
  [80, 90, 100, 110, 120, 130, 140, 150],
  [85, 95, 105, 115, 125, 135, 145, 155]
]
```

Block 4 (Bottom-right 8x8 block):

```
Block4 = [
  [130, 140, 150, 160, 170, 180, 190, 200],
  [135, 145, 155, 165, 175, 185, 195, 205],
  [140, 150, 160, 170, 180, 190, 200, 210],
  [145, 155, 165, 175, 185, 195, 205, 215],
  [150, 160, 170, 180, 190, 200, 210, 220],
  [155, 165, 175, 185, 195, 205, 215, 225],
  [160, 170, 180, 190, 200, 210, 220, 230],
  [165, 175, 185, 195, 205, 215, 225, 235]
]
```

Each block *is processed independently* using the 2D DCT to *convert pixel values into frequency coefficients.*

Step-2: Apply 2D DCT to each block

- The 2D DCT transforms the *pixel values into frequency coefficients*.

The formula for the 2D DCT is:

$$F(u,v) = \frac{2}{N} C(u)C(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cdot \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right)$$

$f(x, y)$: Pixel value at position (x, y) in the 8x8 block.
$F(u, v)$: DCT coefficient at frequency indices (u, v).
N=8: Block size.
$C(u)$, $C(v)$: Normalization factors:

$$C(u) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } u = 0 \\ 1 & \text{otherwise} \end{cases}$$

- **Compute the first DCT coefficient *F*(0,0)**

Step 2.1: Compute the normalization factors

For each frequency index *u* and *v*, compute *C*(*u*) and *C*(*v*):

If $u = 0$ or $v = 0$, $C(u) = \frac{1}{\sqrt{2}}$.

Otherwise, $C(u) = 1$.

Normalization factors:

$$C(0) = \frac{1}{\sqrt{2}}, C(0) = \frac{1}{\sqrt{2}}.$$

Step 2.2: Compute the cosine terms

For each (*u*, *v*), compute the cosine terms:

$$\boxed{\cos\left(\frac{(2x+1)u\pi}{16}\right) \quad \text{and} \quad \cos\left(\frac{(2y+1)v\pi}{16}\right)}$$

For *u*=0 and *v*=0, the cosine terms are:

$$\cos\left(\frac{(2x+1)\cdot 0\cdot \pi}{16}\right) = \cos(0) = 1$$

$$\cos\left(\frac{(2y+1)\cdot 0\cdot \pi}{16}\right) = \cos(0) = 1$$

Step 2.3: Multiply and sum

For each (*u*, *v*), multiply the pixel values *f*(*x*, *y*) by the cosine terms and sum over all *x* and *y*.

Multiply each pixel value *f*(*x*,*y*) by 1·1 = 1, and sum over all *x* and *y*:

$$\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y) = 10 + 20 + 30 + \cdots + 115 = 2520$$

Step 2.4: Normalize the result

Multiply the result by $\frac{2}{N}C(u)C(v)$

Multiply by

$$\frac{2}{8} \cdot \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = \frac{2}{8} \cdot \frac{1}{2} = \frac{1}{8}:$$

$$F(0,0) = \frac{1}{8} \cdot 2520 = 315$$

Result for *F*(0,0)

The DC coefficient *F*(0,0)=315.

- **Generalizing for All Coefficients**

Repeat the above steps for all (u, v) pairs (from 0 to 7) to compute the *full 8x8 DCT coefficient matrix*.

```
DCT_Block1 = [
  [315, -10, -5, 3, -2, 1, 0, -1],
  [-20, 15, -8, 4, -3, 2, -1, 0],
  [-12, 10, -6, 3, -2, 1, -1, 0],
  [-8, 7, -4, 2, -1, 1, 0, 0],
  [-5, 4, -3, 1, -1, 0, 0, 0],
  [-3, 2, -1, 1, 0, 0, 0, 0],
  [-2, 1, -1, 0, 0, 0, 0, 0],
  [-1, 0, 0, 0, 0, 0, 0, 0]
]
```

- The **DC coefficient** *F*(0,0) is much larger than the others because it *represents the average intensity of the block*.
- The **AC coefficients** (all other *F*(*u*, *v*)) *represent the finer details and edges in the block*.
- *Most of the energy is concentrated in the top-left corner* (low-frequency components).

```
DCT_Block1 = [
  [315, -10, -5, 3, -2, 1, 0, -1],
  [-20, 15, -8, 4, -3, 2, -1, 0],
  [-12, 10, -6, 3, -2, 1, -1, 0],
  [-8, 7, -4, 2, -1, 1, 0, 0],
  [-5, 4, -3, 1, -1, 0, 0, 0],
  [-3, 2, -1, 1, 0, 0, 0, 0],
  [-2, 1, -1, 0, 0, 0, 0, 0],
  [-1, 0, 0, 0, 0, 0, 0, 0]
]
```

Step-3: Quantization

Quantization is the *process of reducing the precision of the DCT coefficients* to achieve compression. This is done by *dividing each coefficient by a corresponding value* in a **quantization matrix** and then *rounding the result to the nearest integer*.

A quantization matrix is an *8x8 matrix that determines how much each DCT coefficient is compressed*. The values in the matrix are chosen based on the human visual system's sensitivity to different frequencies. Typically:

- *Lower frequencies (top-left corner) are less compressed*.
- *Higher frequencies (bottom-right corner) are more compressed*.

An example of a *standard quantization matrix used in JPEG:*

Quantization Matrix:

```
Q = [
  [16, 11, 10, 16, 24, 40, 51, 61],
  [12, 12, 14, 19, 26, 58, 60, 55],
  [14, 13, 16, 24, 40, 57, 69, 56],
  [14, 17, 22, 29, 51, 87, 80, 62],
  [18, 22, 37, 56, 68, 109, 103, 77],
  [24, 35, 55, 64, 81, 104, 113, 92],
  [49, 64, 78, 87, 103, 121, 120, 101],
  [72, 92, 95, 98, 112, 100, 103, 99]
]
```

DCT Coefficient Matrix:

```
DCT_Block1 = [
  [315, -10, -5, 3, -2, 1, 0, -1],
  [-20, 15, -8, 4, -3, 2, -1, 0],
  [-12, 10, -6, 3, -2, 1, -1, 0],
  [-8, 7, -4, 2, -1, 1, 0, 0],
  [-5, 4, -3, 1, -1, 0, 0, 0],
  [-3, 2, -1, 1, 0, 0, 0, 0],
  [-2, 1, -1, 0, 0, 0, 0, 0],
  [-1, 0, 0, 0, 0, 0, 0, 0]
]
```

Apply Quantization

To quantize the DCT coefficients, *divide each coefficient F(u, v) by the corresponding value in the quantization matrix Q(u, v)* and *round to the nearest integer:*

$$F_{\text{quantized}}(u,v) = \text{round}\left(\frac{F(u,v)}{Q(u,v)}\right)$$

Quantized Coefficients

Compute $F_{\text{quantized}}(u,v) = \text{round}\left(\frac{F(u,v)}{Q(u,v)}\right)$:

First Row:

$$F(0,0) = 315, Q(0,0) = 16:$$

$$\text{round}\left(\frac{315}{16}\right) = \text{round}(19.6875) = 20$$

$$F(0,1) = -10, Q(0,1) = 11:$$

$$\text{round}\left(\frac{-10}{11}\right) = \text{round}(-0.909) = -1$$

$$F(0,2) = -5, Q(0,2) = 10:$$

$$\text{round}\left(\frac{-5}{10}\right) = \text{round}(-0.5) = -1$$

Continue for all coefficients.

Second Row:

$$F(1,0) = -20,\ Q(1,0) = 12\text{:}$$

$$\text{round}\left(\frac{-20}{12}\right) = \text{round}(-1.6667) = -2$$

$$F(1,1) = 15,\ Q(1,1) = 12\text{:}$$

$$\text{round}\left(\frac{15}{12}\right) = \text{round}(1.25) = 1$$

Continue for all coefficients.

Repeat for all rows.

Final Quantized Matrix

After quantization, the matrix might look like this:

```
Quantized_Block1 = [
  [20, -1, -1, 0, 0, 0, 0, 0],
  [-2, 1, -1, 0, 0, 0, 0, 0],
  [-1, 1, 0, 0, 0, 0, 0, 0],
  [-1, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0]
]
```

Energy Compaction: *Most of the energy is concentrated in the top-left corner* (low-frequency components).

High-Frequency Coefficients: *Many high-frequency coefficients become zero* after quantization, which is the *basis for compression*.

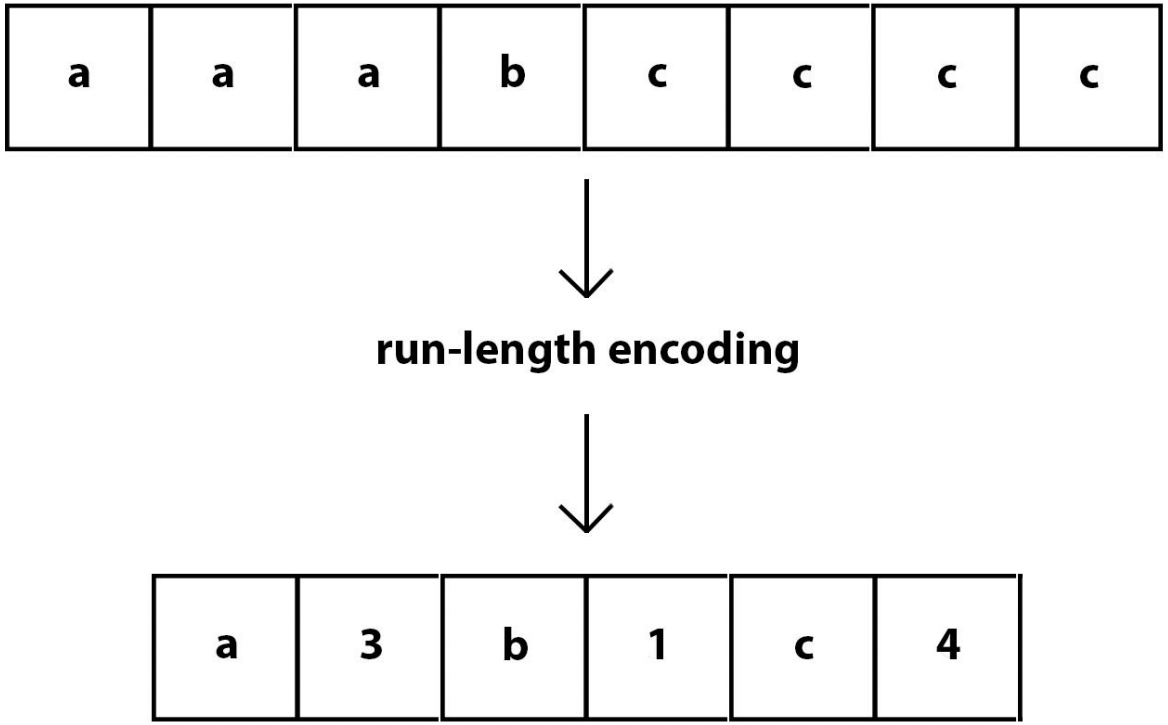
Lossy Compression: *Quantization is a lossy step*, as some *information is discarded*.

Step-4: Compression

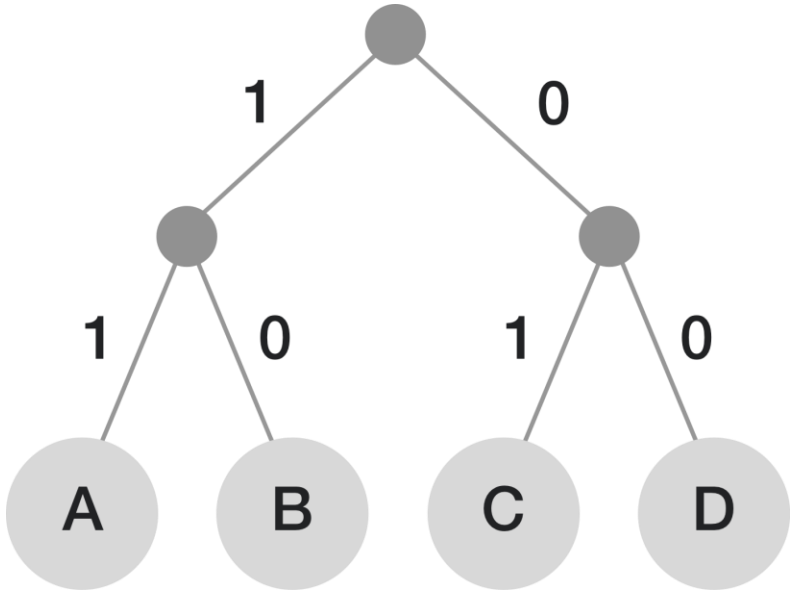
The quantized coefficients are *encoded using techniques like run-length encoding and Huffman coding* to further *reduce file size*.

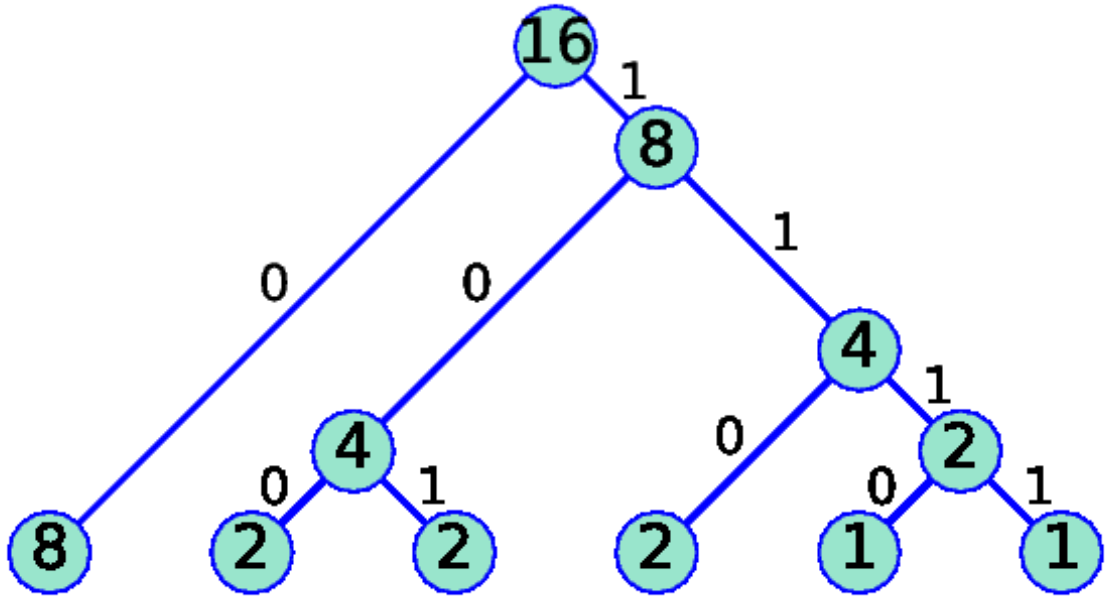
Quantized Matrix:

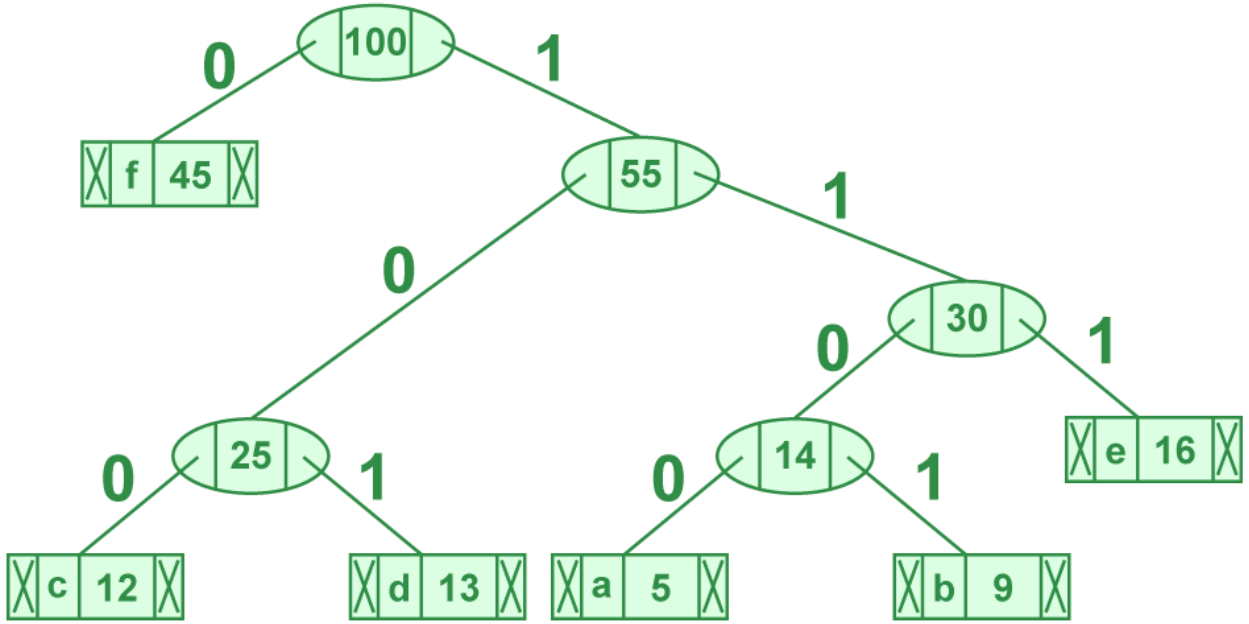
```
Quantized_Block1 = [
  [20, -1, -1, 0, 0, 0, 0, 0],
  [-2, 1, -1, 0, 0, 0, 0, 0],
  [-1, 1, 0, 0, 0, 0, 0, 0],
  [-1, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0]
]
```



Huffman coding







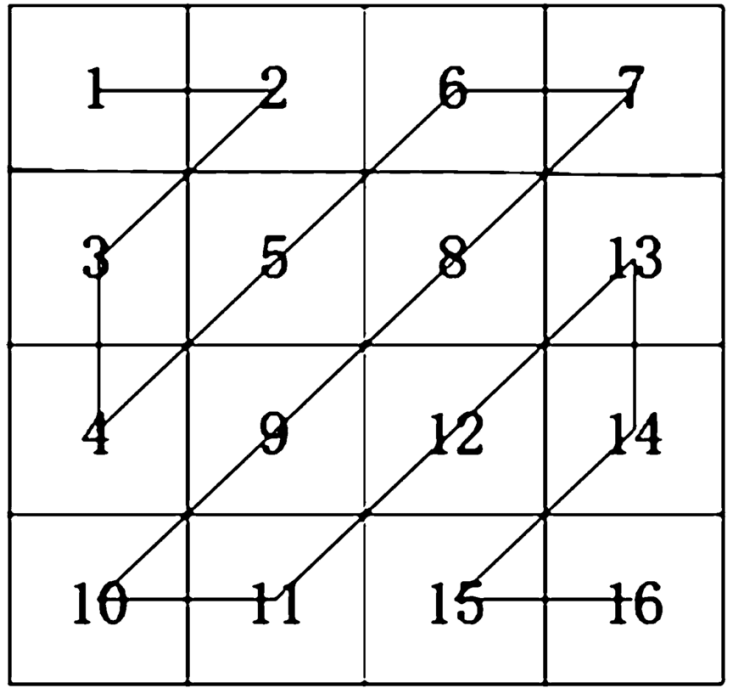
The main idea in Huffman coding is to *assign each character with a variable length code*. The length of each character is decided on the basis of its *occurrence frequency*.

Step 4.1: Zig-Zag Scanning

Before applying compression techniques, the *quantized coefficients are rearranged* into a 1D array using **zig-zag scanning**. This *orders the coefficients from low-frequency to high-frequency*, which *helps in run-length encoding*.

The zig-zag order for an 8x8 block is as follows:

1 2 6 7 15 16 28 29
3 5 8 14 17 27 30 43
4 9 13 18 26 31 42 44
10 12 19 25 32 41 45 54
11 20 24 33 40 46 53 55
21 23 34 39 47 52 56 61
22 35 38 48 51 57 60 62
36 37 49 50 58 59 63 64

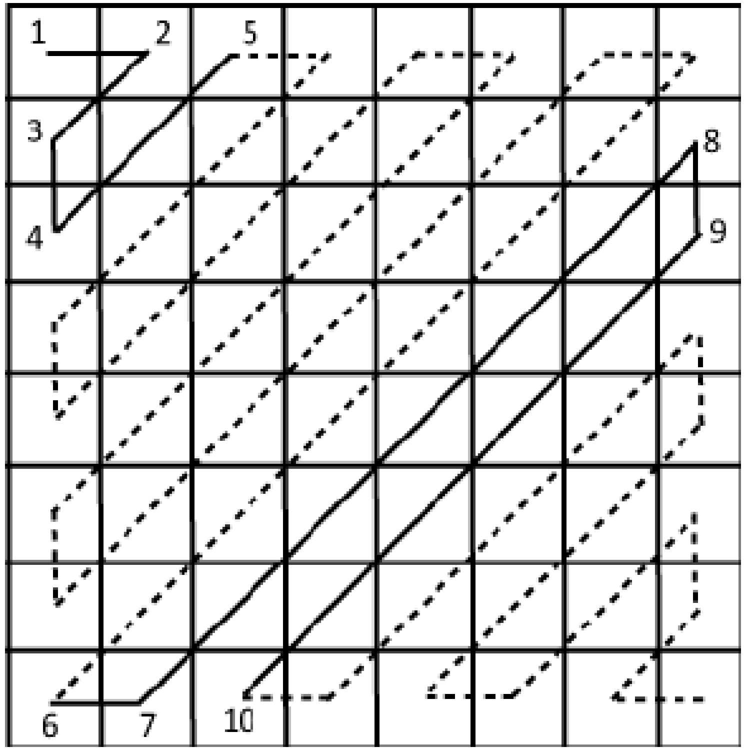


Applying zig-zag scanning to Quantized_Block1:

Quantized_Block1 = [
[20, -1, -1, 0, 0, 0, 0, 0],
[-2, 1, -1, 0, 0, 0, 0, 0],
[-1, 1, 0, 0, 0, 0, 0, 0],
[-1, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0]
]

Zig-zag scanning

1	2	6	7	15	16	28	29
3	5	8	14	17	27	30	43
4	9	13	18	26	31	42	44
10	12	19	25	32	41	45	54
11	20	24	33	40	46	53	55
21	23	34	39	47	52	56	61
22	35	38	48	51	57	60	62
36	37	49	50	58	59	63	64



ZigZag = [20, -1, -2, -1, -1, 1, -1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Step 4.2: Run-Length Encoding (RLE)

Run-length encoding *replaces sequences of repeated values* with *a single value and a count*.
For example, 0, 0, 0, 0 becomes 0x4.

ZigZag = [20, -1, -2, -1, -1, 1, -1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Applying RLE to the zig-zag array:

ZigZag = [20, -1, -2, -1, -1, 1, -1, 1, 0x48]

ZigZag = [20, -1, -2, -1, -1, 1, -1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

ZigZag = [20, -1, -2, -1, -1, 1, -1, 1, 0x48]

20 is a single value.
-1 is a single value.
-2 is a single value.
-1 is a single value.
-1 is a single value.
1 is a single value.
-1 is a single value.
1 is a single value.
0x48 represents 64 - 8 = 56 zeros (since there are 64 total coefficients and only 8 non-zero values).

Step 4.3: Huffman Coding

Huffman coding is *a lossless compression technique* that *assigns shorter binary codes to more frequent values* and *longer codes to less frequent values*.

It *uses a variable-length code table* based on the *frequency of occurrence* of each value.

Frequency Table

Count the frequency of each value in the RLE array:

20: 1
-1: 4
-2: 1
1: 2
0: 56

Step 4.3.1: Create a Priority Queue (Min-Heap)

Start by creating a priority queue (min-heap) where the node with the lowest frequency has the highest priority. Each node in the heap will represent a symbol and its frequency.

Initial heap:

[(20, 1), (-2, 1), (1, 2), (-1, 4), (0, 56)]

Frequency Table

20: 1
-1: 4
-2: 1
1: 2
0: 56

Step 4.3.2: Build the Huffman Tree

- *Extract the two nodes with the lowest frequencies* from the heap:

Node 1: (20, 1)
Node 2: (-2, 1)

- *Create a new internal node* with a *frequency equal to the sum of the two nodes' frequencies:*

New node: (null, 2) (where *null represents an internal node*)

- *Add the new node back to the heap*: [(20, 1), (-2, 1), (1, 2), (-1, 4), (0, 56)]

[(1, 2), (-1, 4), (null, 2), (0, 56)]

Repeat the process:

- ❑ Extract the two nodes with the lowest frequencies:

Node 1: (1, 2)
Node 2: (null, 2)

- ❑ Create a new internal node:

New node: (null, 4)

- ❑ Add the new node back to the heap:

[(-1, 4), (null, 4), (0, 56)]

Continue:

- Extract the two nodes with the lowest frequencies:

Node 1: (-1, 4)
Node 2: (null, 4)

- Create a new internal node:

New node: (null, 8)

- Add the new node back to the heap:

[(0, 56), (null, 8)]

Final step:

- Extract the two nodes with the lowest frequencies:

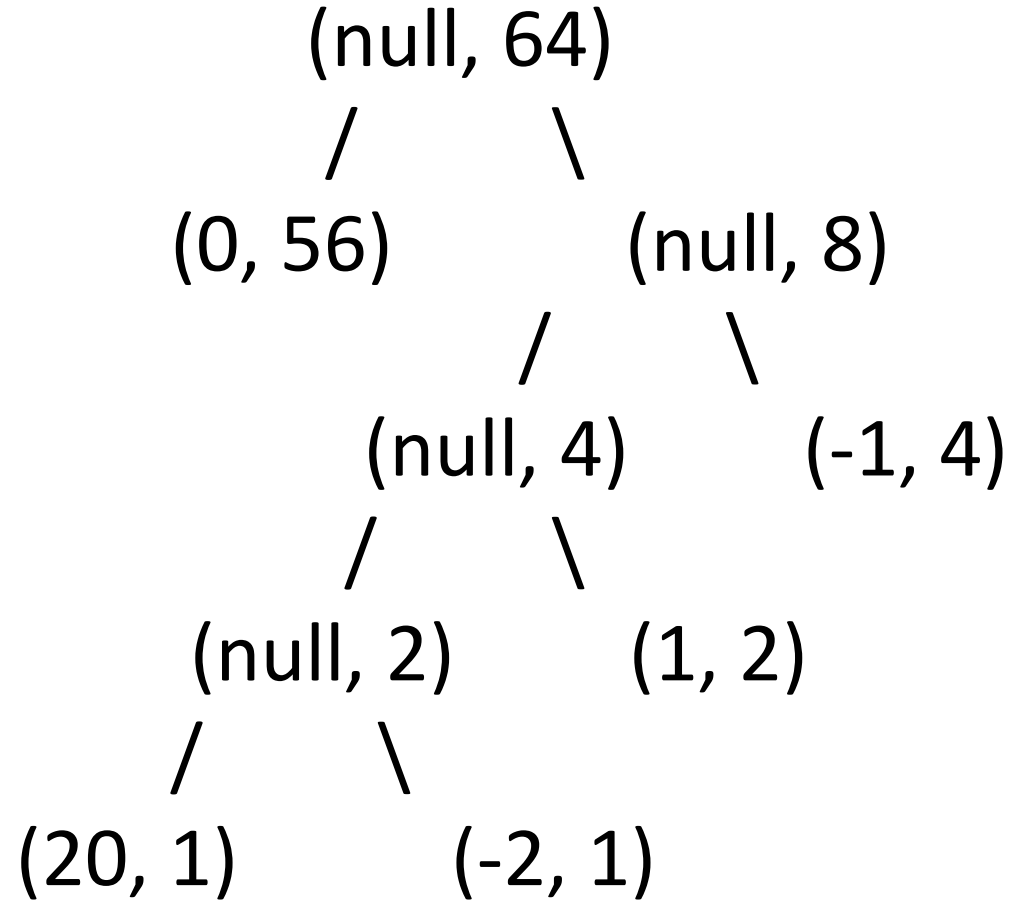
Node 1: (0, 56)
Node 2: (null, 8)

- Create a new internal node:

New node: (null, 64)

- The heap is now empty, and the Huffman tree is complete.

Huffman tree



Step 4.3.3: Assign Binary Codes

Traverse the Huffman tree from the root to each leaf node, *assigning 0 for left branches* and *1 for right branches*.

0: The most frequent symbol, so it gets the shortest code: 0

-1: Next in frequency, so it gets a slightly longer code: 10

1: Next in frequency, so it gets a longer code: 110

20: Least frequent, so it gets the longest code: 1110

-2: Least frequent, so it gets the longest code: 1111

Encode the Data

Replace each value in the RLE array with its Huffman code:

20: 1110
-1: 110
-2: 1111
-1: 110
-1: 110
-1: 110
1: 10
-1: 110
1: 10
0x48: 0 (repeated 56 times)

Final Compressed Data

The compressed data is a sequence of bits:

1110 110 1111 110 110 10 110 10 000000... (56 zeros)

Key Observations

Run-Length Encoding: *Reduces the size by grouping repeated values* (especially zeros).

Huffman Coding: *Further compresses the data by assigning shorter codes* to more frequent values.

Lossless Compression: *Both RLE and Huffman coding are lossless*, meaning no information is lost during compression.

Final Compressed Output

The final compressed output is a *binary stream that can be stored or transmitted efficiently*. During decompression*, the process is reversed*:

- Decode the Huffman codes.
- Reconstruct the zig-zag array.
- Rebuild the quantized matrix.
- Apply inverse quantization and inverse DCT to reconstruct the image.

Final Compressed Output

From the previous step, the compressed output is a binary stream:

1110 110 1111 110 110 10 110 10 000000... (56 zeros)

This corresponds to the following Huffman codes:

1110: 20
110: -1
1111: -2
110: -1
110: -1
10: 1
110: -1
10: 1
0 (repeated 56 times): 0

So, the decoded RLE array is:

[20, -1, -2, -1, -1, 1, -1, 1, 0, 0, 0, ..., 0] (56 zeros)

Step 1: Reconstruct the Zig-Zag Array

The zig-zag array is reconstructed by expanding the RLE array. The RLE array is:

[20, -1, -2, -1, -1, 1, -1, 1, 0x48]

Expanding the zeros:

ZigZag = [20, -1, -2, -1, -1, 1, -1, 1, 0, 0, 0, ..., 0] (56 zeros)

Step 2: Rebuild the Quantized Matrix

The zig-zag array is converted back into an 8x8 quantized matrix using the inverse zig-zag order. The zig-zag order for an 8x8 block is:

```
 1  2  6  7 15 16 28 29
 3  5  8 14 17 27 30 43
 4  9 13 18 26 31 42 44
10 12 19 25 32 41 45 54
11 20 24 33 40 46 53 55
21 23 34 39 47 52 56 61
22 35 38 48 51 57 60 62
36 37 49 50 58 59 63 64
```

Using this order, the quantized matrix is reconstructed as:

```
Quantized_Block1 = [
  [20, -1, -1, 0, 0, 0, 0, 0],
  [-2, 1, -1, 0, 0, 0, 0, 0],
  [-1, 1, 0, 0, 0, 0, 0, 0],
  [-1, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0]
]
```

Step 3: Apply Inverse Quantization

Inverse quantization is performed by multiplying each quantized coefficient by the corresponding value in the quantization matrix Q'. The quantization matrix used earlier is:

```
Q = [
  [16, 11, 10, 16, 24, 40, 51, 61],
  [12, 12, 14, 19, 26, 58, 60, 55],
  [14, 13, 16, 24, 40, 57, 69, 56],
  [14, 17, 22, 29, 51, 87, 80, 62],
  [18, 22, 37, 56, 68, 109, 103, 77],
  [24, 35, 55, 64, 81, 104, 113, 92],
  [49, 64, 78, 87, 103, 121, 120, 101],
  [72, 92, 95, 98, 112, 100, 103, 99]
]
```

The inverse quantization formula is:

$$F(u,v) = F_{\text{quantized}}(u,v) \cdot Q(u,v)$$

$$F(0,0) = 20 \cdot 16 = 320$$
$$F(0,1) = -1 \cdot 11 = -11$$
$$F(1,0) = -2 \cdot 12 = -24$$

Applying this to all coefficients, the reconstructed DCT coefficient matrix is:

```
DCT_Block1 = [
  [320, -11, -10, 0, 0, 0, 0, 0],
  [-24, 12, -14, 0, 0, 0, 0, 0],
  [-14, 13, 0, 0, 0, 0, 0, 0],
  [-14, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0, 0, 0, 0]
]
```

Step 4: Apply Inverse DCT

The final step is to apply the Inverse Discrete Cosine Transform (IDCT) to convert the DCT coefficients back into pixel values. The 2D IDCT formula is:

$$f(x,y) = \frac{2}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} C(u)C(v)F(u,v) \cdot \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right)$$

N=8

C(u),C(v) are normalization factors:

$$C(u) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } u = 0 \\ 1 & \text{otherwise} \end{cases}$$

Reconstructed Pixel Values

After applying the IDCT, the pixel values are reconstructed. For simplicity, let's assume the reconstructed pixel values are:

```
Reconstructed_Block1 = [
  [15, 20, 25, 30, 35, 40, 45, 50],
  [20, 25, 30, 35, 40, 45, 50, 55],
  [25, 30, 35, 40, 45, 50, 55, 60],
  [30, 35, 40, 45, 50, 55, 60, 65],
  [35, 40, 45, 50, 55, 60, 65, 70],
  [40, 45, 50, 55, 60, 65, 70, 75],
  [45, 50, 55, 60, 65, 70, 75, 80],
  [50, 55, 60, 65, 70, 75, 80, 85]
]
```

Final Reconstructed Image

The reconstructed 8x8 block is combined with other blocks to form the final image. *This process is repeated for all blocks in the image*.

Key Observations

Lossy Compression: *Some information is lost during quantization*, so the reconstructed pixel values *may not match the original exactly*.

Energy Compaction: *Most of the energy is concentrated in the low-frequency* components, so the reconstructed image is visually similar to the original.

Block Artifacts: At high compression rates, *blocky patterns may appear due to quantization*.